

Учебный конспект к защите
Математическое моделирование и реализация
планировщика для гетерогенных CPU (VCG / scx_auction)

s4 модель + реализация + презентация 21.04

Для: Шаго П. Е. (докладчик)

21 апреля 2026 г.

Содержание

1	Как читать этот документ	3
2	Часть I. Модель системы (s4, §1)	3
2.1	Аппаратная платформа (§1.1)	3
2.2	Задачи-агенты (§1.2)	3
2.3	Бюджеты (§1.3)	4
3	Часть II. MDP-формулировка (s4, §2)	8
3.1	Пространство состояний (§2.1)	8
3.2	Пространство действий (§2.2)	8
3.3	Переход (§2.3)	9
3.4	Вознаграждения (§2.4)	11
4	Часть III. Контракты и ценообразование (s4, §3)	11
4.1	Простой контракт (§3.1)	11
4.2	Стоимости ядер (§3.2)	11
4.3	Полезность задачи (§3.3)	12
5	Часть IV. Социальная функция стоимости (s4, §4)	12
5.1	Социальное благосостояние (§4.1)	12
5.2	Социальная стоимость (§4.2)	12
6	Часть V. VCG-механизм (s4, §5)	12
6.1	Offline, полная информация (§5.1)	12
6.2	Адаптация к гетерогенным ядрам (§5.2)	13
6.3	Виртуальные оценки (§5.3, Sano)	13
7	Часть VI. Протокол аукциона (s4, §6)	13
7.1	На каждом кванте (§6.1)	13
7.2	Платёж VCG (§6.2)	14
7.3	Дифференцированное ценообразование P vs E (§6.3)	14
8	Часть VII. Онлайн-обучение (s4, §7)	14
8.1	ИHMDP-VCG (Leon & Etesami 2022)	14
8.2	Обновление политики (§7.2)	15
8.3	Гарантии (§7.3)	15

9	Часть VIII. Свойства механизма (s4, §8)	15
9.1	Incentive compatibility (§8.1, Sano Thm 1)	15
9.2	IR (§8.2)	15
9.3	Бюджетная допустимость (§8.3)	15
10	Часть IX. Пример: бинарные типы, одно свободное ядро (s4, §9)	16
10.1	Сводка обозначений	16
11	Часть X. Реализация: отличия от теории (VCG_s4_impl.md)	17
11.1	I1. Непрерывная ϕ (отказ от квантования)	17
11.2	I2. Роутинг по бюджетному отношению с гистерезисом	17
11.3	I3. Пропорциональная задержка для «голодающих»	17
11.4	I4. Платёж VCG в форме Vickrey reserve	18
11.4.1	I4.1. Оценщик φ_{hi} на кластер	18
11.4.2	I4.2. Формула платежа	18
11.4.3	I4.3. Свойства	18
11.5	I5. Wake-latency boost (VCG бюджет $\leftrightarrow$ EEVDF дедлайн)	18
11.6	I6. Порядок dispatch и cross-cluster work-stealing	19
11.7	I7. Work-conservation overrides	19
11.8	I8. Параметры реализации	20
11.9	I9. Соответствие код $\rightarrow$ модель	20
12	Часть XI. Презентация 21.04 — по слайдам	21
13	Часть XII. Частые вопросы комиссии — шпаргалка	25

1 Как читать этот документ

Документ — «шпаргалка докладчика». Для каждой формулы и понятия из s4 модели (models/VCG_s4.md), реализации (models/VCG_s4_impl.md) и презентации 21.04 (events/2104/main.tex) дано:

- что это формально;
- почему она выглядит именно так (экономический и MDP-смысл);
- откуда взялась (ссылка на источник — Leon & Etesami, Sano, Myerson, EEVDF);
- как это отображается в коде BPF-планировщика.

[

Главная идея всей работы в одном абзаце] Гетерогенный CPU (P-ядра + E-ядра) — это рынок. Кванты времени — скоропортящийся товар (не использовал — потерял). Задачи — агенты с частным типом $\theta_i = (v_i, l_i)$: ценностью выполнения и длиной. Планировщик — аукционист. Каждый квант он решает кому дать ядро и какого типа (P или E), максимизируя социальное благосостояние. Плата с победителя — VCG (Vickrey–Clarke–Groves): задача платит экстерналию, которую создаёт для остальных. Это делает правдивую ставку доминирующей стратегией.

2 Часть I. Модель системы (s4, §1)

2.1 Аппаратная платформа (§1.1)

Два класса ядер:

$$\mathcal{P} = \{P_1, \dots, P_{K_P}\}, \quad \mathcal{E} = \{E_1, \dots, E_{K_E}\}, \quad K = K_P + K_E.$$

Время дискретно: $t = 1, 2, 3, \dots$ — тики планировщика (кванты). На каждом шаге каждое ядро — скоропортящийся товар: не использованный квант теряется.

[

Почему «скоропортящийся товар»? Это не метафора — это техническая реальность. CPU в состоянии idle этот тик просто простаивает; welfare-потеря строго положительна. Из этого следует важное свойство реализации (§I6): безусловное work-stealing между кластерами — лучше пустить задачу E на P-ядро, чем оставить P пустым.

2.2 Задачи-агенты (§1.2)

Множество активных задач $\mathcal{N}^t$. Новые задачи приходят стохастически: N_t — i.i.d. случайная величина.

Приватный тип задачи i :

$$\theta_i = (v_i, l_i) \in \Theta, \quad v_i \in [0, \bar{V}], \quad l_i \in \{1, \dots, L\}.$$

v_i — ценность полного выполнения, l_i — длина в квантах P-ядра.

ISA (набор инструкций) общий — любая задача исполнима на любом ядре. Разница P vs E только в стоимости кванта и скорости. Выбор типа ядра — решение аукциона, не задачи.

Ценность v_i начисляется только при полном выполнении за l_i подряд идущих квантов (условие «full completion» из Sano 2023). На E-ядре длина масштабируется:

$$l_i^{(E)} = \lceil l_i \cdot \sigma \rceil, \quad \sigma = \eta_P / \eta_E > 1.$$

Здесь η_κ — производительность ядра типа κ .

Зачем «full completion»] Без этого условия задача могла бы «срывать куш» после первого же кванта. «Full completion» делает динамику нетривиальной: контракт на m квантов — это настоящий контракт, и агент несёт риск его невыполнения ($\rightarrow$ неотрицательная полезность требует механизма, а не просто жадного распределения).

2.3 Бюджеты (§1.3)

Что такое бюджет. $B_i > 0$ — покупательная способность задачи i в валюте аукциона. Не «квота времени», не «приоритет в очереди», а именно сумма, которую задача может потратить на платежи VCG. Аналог виртуальных денег или эфирного газа.

Зачем он нужен. Классический VCG — quasi-linear: задача платит столько, сколько требует pivot-правило, и у неё всегда есть чем платить. В операционной системе это опасно — один злой процесс может навязать себе высокий v_i и перекрыть всю welfare. Бюджет = hard limit на экстернальности. Это превращает VCG из одноразового аукциона в повторяющийся с ограничением ресурса. Без B_i механизм не был бы DoS-устойчив.

Начальное значение по классу обслуживания.

Класс	Обозн.	Типичный масштаб	Смысл
Real-time	B_{RT}	$B_{RT} \gg 1$	Гарантированное исполнение, почти неограничен
Interactive	B_{int}	$\sim B_i^{\max}$	Высокий приоритет (GUI, IO-bound)
Batch	B_{batch}	$< B_i^{\max}$	Низкий приоритет (компиляция, фон)
Background	B_{bg}	≈ 0	Только при наличии свободного ресурса

В реализации (§I8) классы отображаются через $p \rightarrow scx.weight$ (Linux nice-value): $B_i^{\max} = w_i \cdot \text{BUDGET_MUL}$, $\text{BUDGET_MUL}=2000$. То есть бюджет линейно зависит от nice: nice 0 ($w = 1024$) даёт $B^{\max} \approx 2 \cdot 10^6$, nice -20 ($w = 88761$) даёт $B^{\max} \approx 1.8 \cdot 10^8$.

Бюджетное ограничение.

$$\sum_{s=t}^{t+m_i-1} p_i^s \leq B_i.$$

Словами: сумма всех платежей за всё время контракта задачи i не превышает её начального бюджета. Если в какой-то момент t' остаток $b_i^{t'}$ меньше требуемого $p_i^{t'}$ — задача не получает ядро в этом кванте (§8.3).

Почему сумма, а не на каждом кванте] Контракт m_i квантов длинный. Платежи p_i^s per-quantum могут колебаться в зависимости от загрузки системы (второй конкурент сильнее/слабее). Ограничение на сумму означает: задача платит «общую смету», без жёсткого потолка на отдельный платёж. Это даёт гибкость — в пиковый момент можно заплатить больше, если есть запас; в тихий момент — меньше. Аналог: кредитка с лимитом, а не дневная норма.

Динамика бюджета (связь с §2.3). Бюджет — компонент состояния $\mathbf{b}^t$. Обновляется каждый квант по правилу:

$$b_i^{t+1} = \min(B_i^{\max}, b_i^t - p_i^t + \Delta_i^t).$$

Три компоненты по порядку важности:

(А) $-p_i^t$ — трата на выигранный квант. Задача заплатила механизму VCG-платёж за этот тик. Если $a_i^t = (0, 0)$ (не получила ядро) — $p_i^t = 0$, баланс не падает. Если получила — $p_i^t > 0$ (в реализации §I4 минимум $c_\kappa \cdot t_{\text{consumed}}/s$, то есть пол = posted-price; может быть больше, если есть pivot через φ_{hi}).

(В) $+\Delta_i^t$ — восстановление за простой.

$$\Delta_i^t = \frac{\Delta t_{\text{idle}} \cdot v_i}{D_{\text{rep}}}, \quad \Delta t_{\text{idle}} = \text{нс, проведённые в состоянии sleep.}$$

Рабочий пример: задача спит 10 мс (10^7 нс), nice 0 ($v_i = 1024$), $D_{\text{rep}} = 5 \cdot 10^6$ нс. Тогда $\Delta = 10^7 \cdot 1024 / (5 \cdot 10^6) = 2048$.

(С) $\min(B_i^{\text{max}}, \cdot)$ — потолок. Бюджет не растёт выше начального $B_i^{\text{max}} = w_i \cdot 2000$. Иначе долго спящий процесс накопил бы «бесконечный кредит» и стал бы neulimitedly приоритетным при пробуждении. Потолок — якорь.

Хронологическая последовательность внутри одного кванта.

1. Наблюдение s^t : текущий b_i^t известен.
2. Механизм решает a^t . При этом проверяется бюджет: если p_i^t (оценка) $> b_i^t$, задача исключается из допустимых (§8.3).
3. Выигравшая задача исполняется на ядре, тратит слайс.
4. На auction_stopping считается фактический $p_i^t = \max(c_\kappa t_c/s, \varphi_{\text{hi}} t_c/s^2)$ (§I4.2), списывается: $b_i \leftarrow p_i^t$.
5. Если задача уходит в sleep (блокировка на IO) — при следующем wake считается Δt_{idle} и начисляется Δ_i .
6. Сложение и обрезка потолком $\rightarrow b_i^{t+1}$.

Почему это именно такое правило.

[

Почему восстановление $\propto v_i$] Без этого высокоприоритетная задача, застрявшая в IO на 50 мс, пришла бы к моменту wake с тем же Δ , что и nice-19 batch. Пропорциональность v_i сохраняет классовое различие: за равное время простоя interactive-задача восстанавливает больше, чем background. Аналог: процент по вкладу зависит от суммы.

[

Откуда берётся $D_{\text{rep}} = 5$ мс] REPLENISH_DIV = $5 \cdot 10^6$ нс совпадает с SCX_SLICE_DFL (дефолтный слайс EEVDF). Смысл: за один слайс простоя задача $v_i = 1024$ получает $\Delta \approx 1024$. Это одна единица посторонней платы порядка c_κ . Иначе говоря — «за слайс сна зарабатывается на покупку слайса выполнения». Баланс tune-нут так, что равномерно работающая задача держит $\rho_i \approx 0.5$.

[

Почему списание и пополнение в одном такте] Можно было бы — два разных тика. Но тогда появилась бы гасе: задача могла бы «спать» между списанием и следующим пополнением и не получить заслуженного Δ . Единое обновление $b^{t+1} = b^t - p + \Delta$ делает бухгалтерию атомарной — важно для BPF, где пересечение хуков по CPU может быть недетерминированным.

Инварианты.

- $0 \leq b_i^t \leq B_i^{\max}$ всегда (левое — из §8.3 запрета win при нехватке, правое — из потолка).
- $\rho_i^t = b_i^t / B_i^{\max} \in [0, 1]$, определён всегда.
- Если задача никогда не исполнялась — $b_i^t = B_i^{\max}$ (новая задача входит с полным кошельком).
- Если задача CPU-hog — b_i^t монотонно убывает до 0 (становится STARVED), после чего механизм её демотивирует.

Связь со состоянием MDP (§2.1, §2.3). Компонент $\mathbf{b}^t$ — не «внутренняя переменная», а часть состояния. Значит политика π может смотреть на бюджеты всех задач и учитывать их в $\arg\max W$. Именно поэтому кандидаты с низким b_i допустимы в споре действий, но их ϕ_κ фактически срезается бюджетным ограничением — они не выигрывают, даже если v_i высок.

В реализации (§I2) это выражается иначе: бюджет используется не напрямую в ϕ , а через ρ_i для роутинга в DSQ. Это аппроксимация: вместо решения полной задачи с бюджетным ограничением — грубое «куда отправить». Равновесие поддерживается штрафом STARVED для задач с малым ρ .

Роль в роутинге. Определение сигнала.

$$\rho_i^t = \frac{b_i^t}{B_i^{\max}} \in [0, 1].$$

Нормированная покупательная способность. $\rho = 1$ — «полный кошелёк»; $\rho = 0$ — «банкрот». По построению ρ растёт во время простоя и падает во время активного исполнения.

Почему ρ — поведенческий сигнал. Из формулы $b_i^{t+1} = b_i^t - p_i^t + \Delta_i^t$: знак Δb определяется балансом $-p_i^t + \Delta_i^t$. На большом окне времени:

$$\frac{\text{траты на CPU}}{\text{время простоя}} \sim \frac{p_i}{\Delta_i} \sim \frac{\text{active}}{\text{idle}} \cdot \text{const.}$$

Значит: ρ_i — прокси для обратного отношения active/idle. Высокий $\rho \Leftrightarrow$ много idle $\Leftrightarrow$ interactive. Низкий $\rho \Leftrightarrow$ много active $\Leftrightarrow$ CPU-bound. Это получается само собой из бухгалтерии — никто явно не классифицировал процесс.

Правила роутинга (реализация, §I2 с гистерезисом).

Условие	DSQ	Интерпретация
$\rho_i \geq \rho_H = 0.70$	AUCTION_DSQ_P	bursty, интерактивная, latency-критична
$\rho_i < \rho_L = 0.30$	AUCTION_DSQ_E	CPU-bound, устойчиво расходует
$\rho_L \leq \rho_i < \rho_H$ (deadband)	как было (sticky)	неопределённость — не двигаем
$\rho_i < 1/\text{STARVE_FRAC} = 0.10$	AUCTION_DSQ_STARVED	почти банкрот, штрафуются

[

Почему именно 30/70, а не 50/50] Один порог 50% даёт flapping: задача около середины при мелких колебаниях ρ скачет P $\leftrightarrow$ E, вызывая миграции и потерю кэша. Двусторонний порог 30/70 с deadband [30, 70] реализует threshold policy with inaction region (Sano, контрактные аукционы с одним параметром). В зоне deadband решение «не менять» — дешевле, чем угадывать.

Почему 10% — порог STARVED] STARVE_FRAC= 10 означает «при $b \leq B^{\max}/10$ задача считается банкротом». Это ниже ρ_L — то есть сначала задача переходит в E-кластер (проще и дешевле), и только если продолжает тратить — падает в STARVED. Двухстадийная защита от DoS.

Что происходит в STARVED. Задача не отбрасывается — она демотивируется: виртуальный дедлайн сдвигается на $\max(0, -\phi_\kappa)/D_{\text{starve}}$ (§I3), то есть её выполнят после нормальных задач. Это аналог «задержки в очереди», а не kill. Пока задача ждёт — её бюджет пополняется через idle- Δ , и она постепенно возвращается к $\rho \geq 0.10$.

Бюджет как двойной объект. Один и тот же B_i обслуживает две разные задачи механизма, без дополнительных структур:

Задача	Как используется B	Где в теории
Защита от DoS	жёсткое огр. $\sum p \leq B$	§1.3, §8.3
Классификация задачи	ρ -рутинг в DSQ	§I2 реализации

Красота этого совмещения] Нормальный OS-планировщик держит отдельные структуры: priority (для очереди), load average (для IO-vs-CPU классификации), rate limit (для anti-hog). У нас одно поле решает всё три задачи одновременно: приоритет = v_i (вход), позиция в классе = ρ_i (динамика), anti-hog = порог STARVED. Это механизм-дизайн, а не инженерный хак: бюджет получается многофункциональным потому что он — валюта, а не число.

Почему ρ , а не абсолютный b] Нормировка на $B_i^{\max}$ убирает разницу классов: nice 0 и nice -20 имеют разные $B^{\max}$ в сотни раз, но одинаковые ρ означают одинаковое поведенческое состояние («половинный кошелек» для обоих = средняя активность). Рутинг должен зависеть от поведения, не от класса — иначе все nice-20 ушли бы в P автоматически, независимо от их IO-профиля.

Разбор трёх сценариев.

Сценарий	Δ_i	p_i	Куда движется ρ_i
CPU-hog (tight loop)	0	> 0 каждый квант	падает $\rightarrow 0$ (starve)
Interactive (wait IO, then run)	> 0 долго	> 0 коротко	растёт к 1 (P-кластер)
Sleeper (почти не просыпается)	> 0	≈ 0	= 1 (насыщен)

Связь с Myerson-резервом] Прирост бюджета играет роль порогового правила: задача, накопившая достаточно, «проходит» в аукцион. В одномерных контрактных моделях это называется threshold policy: агент допускается к торгу, когда его виртуальная оценка $\varphi(\theta_i)$ пересекает порог. Здесь порог реализован через ρ_i — сравнение накопленного бюджета с максимумом.

3 Часть II. MDP-формулировка (s4, §2)

3.1 Пространство состояний (§2.1)

$$s^t = (\mathbf{x}^t, \mathbf{b}^t, \boldsymbol{\omega}^t)$$

- $\mathbf{x}^t = (x_1^t, \dots, x_K^t) \in \{0, \dots, L-1\}^K$ — оставшиеся кванты «аренды» на каждом ядре ($x_k^t = 0$ означает «ядро свободно»);
- $\mathbf{b}^t$ — остатки бюджетов активных задач;
- $\boldsymbol{\omega}^t$ — внешние факторы (температура, энергопотребление, память).

Свободные ядра:

$$K_P^t = K_P - |\{k \in \mathcal{P} : x_k^t \geq 1\}|, \quad K_E^t = K_E - |\{k \in \mathcal{E} : x_k^t \geq 1\}|.$$

3.2 Пространство действий (§2.2)

Что такое действие. Действие a^t на шаге t — это решение механизма о распределении всех свободных ядер по всем активным задачам. Не одной задаче, не одному ядру: всё сразу. Это decision variable, которую оптимизирует планировщик, максимизируя welfare.

Формальная запись. Для каждой задачи $i \in \mathcal{N}^t$ вводится пара индикаторов:

$$a_i^t = (a_{i,P}^t, a_{i,E}^t) \in \{0, 1\}^2.$$

Смысл битов:

- $a_{i,P}^t = 1$ — «на шаге t задача i получает какое-то Р-ядро»;
- $a_{i,E}^t = 1$ — «получает какое-то Е-ядро»;
- $a_{i,P}^t = a_{i,E}^t = 0$ — «не получает ничего в этот квант».

[

Почему не указывается какое именно ядро] Внутри кластера ядра симметричны — Р1 и Р2 отличаются только номером. С точки зрения модели выдача задаче Р1 или Р2 даёт тот же ϕ_P , ту же будущую welfare. Поэтому действие агрегирует: важен класс ядра, не его индекс. Конкретное ядро выбирается уже отдельно (в реализации — через `scx_bpf_pick_idle_cpu` с учётом NUMA/cache locality, §I7.1).

Взаимное исключение: задача на одном ядре.

$$a_{i,P}^t + a_{i,E}^t \leq 1, \quad \forall i \in \mathcal{N}^t.$$

Словами: одна задача в один квант занимает не более одного ядра. Нет параллелизма внутри задачи (модель single-threaded: одна задача $\equiv$ один поток). Если задача многопоточная — в модели она представлена несколькими $i_1, i_2, \dots$, по одному на поток.

[

Почему не $= 1$, а ≤ 1] ≤ 1 позволяет $a_{i,P}^t = a_{i,E}^t = 0$ — задача может вообще ничего не получить на этом шаге (ядер не хватило, либо механизм решил, что её ϕ слишком мал по сравнению с конкурентами). Строгое равенство потребовало бы, чтобы каждой активной задаче выделялось ядро каждый квант, что невыполнимо при $|\mathcal{N}^t| > K$.

Ёмкостные ограничения: не превысить число свободных ядер.

$$\sum_{i \in \mathcal{N}^t} a_{i,P}^t \leq K_P^t, \quad \sum_{i \in \mathcal{N}^t} a_{i,E}^t \leq K_E^t.$$

Словами: суммарное число задач, получающих Р-ядро, не превышает числа свободных Р-ядер K_P^t (аналогично для Е). Напомним: K_P^t считается по состоянию $\mathbf{x}^t$ — занятые задачами из прошлых квантов (ещё досматривающие свой контракт) ядра не свободны, даже если сейчас мог бы быть лучший победитель.

Полное пространство действий.

$$\mathcal{A}(s^t) = \left\{ (a_i^t)_{i \in \mathcal{N}^t} : a_i^t \in \{0, 1\}^2, a_{i,P}^t + a_{i,E}^t \leq 1, \sum_i a_{i,P}^t \leq K_P^t, \sum_i a_{i,E}^t \leq K_E^t \right\}.$$

Мощность: верхняя оценка $|\mathcal{A}(s^t)| \leq 3^{|\mathcal{N}^t|}$ (каждая задача: Р, Е или пропуск), но с учётом ограничений — существенно меньше.

[

Зачем ограничение именно на свободные, а не на все] Занятое ядро ещё исполняет прошлый контракт (simple contract, §3.1: раз выдали — гарантировано). Переназначить его сейчас — нарушить контракт и сломать IC (задача не сможет рассчитывать на своё обещанное время). Поэтому «свободные» K_P^t, K_E^t , а не «всего» K_P, K_E .

Пример. $K_P = 2, K_E = 3, \mathbf{x}^t = (0, 2, 0, 0, 1)$ (ядра P_1, E_1, E_2 свободны, P_2, E_3 заняты). Значит $K_P^t = 1, K_E^t = 2. \mathcal{N}^t = \{1, 2, 3, 4\}$, четыре задачи конкурируют.

Пример допустимого действия:

$$a_1^t = (1, 0), a_2^t = (0, 1), a_3^t = (0, 1), a_4^t = (0, 0).$$

Задача 1 получает Р ($\sum a_{\cdot,P} = 1 \leq 1$ ОК), задачи 2 и 3 — Е ($\sum a_{\cdot,E} = 2 \leq 2$ ОК), задача 4 пропускает этот квант.

Пример недопустимого: $a_1 = (1, 1)$ — задача получила и Р, и Е одновременно (нарушение $a_{1,P} + a_{1,E} \leq 1$).

Как механизм выбирает a^t . Оптимальное действие — $\arg\max$ социального благосостояния:

$$a^{t*} = \arg \max_{a \in \mathcal{A}(s^t)} \sum_i [a_{i,P} \phi_P(\theta_i) + a_{i,E} \phi_E(\theta_i)] + \delta \cdot \mathbb{E}[W(s^{t+1})].$$

Первая сумма — мгновенный welfare (ценность минус стоимость для выигравших задач). Второй член — ожидаемая будущая welfare с дисконтом δ : выдать ядро длинной задаче сейчас = заблокировать его от коротких, которые могли бы прийти потом.

В реализации (§I2, §I7) этот $\arg\max$ не решается точно — заменяется на локальные правила: ρ -роутинг в кластер + EEVDF-ordering внутри DSQ (аппроксимация $\arg\max$ через virtual deadline).

3.3 Переход (§2.3)

Функция перехода:

$$s^{t+1} = G(s^t, a^t, N_{t+1}).$$

Три аргумента: текущее состояние s^t , решение механизма a^t , и случайный приход новых задач N_{t+1} . G детерминирована по первым двум, стохастика целиком в N_{t+1} .

Разложим переход покомпонентно — состояние $s^t = (\mathbf{x}^t, \mathbf{b}^t, \omega^t)$.

(1) Вектор оставшихся квантов $\mathbf{x}^{t+1}$. Для каждого ядра $k \in \{1, \dots, K\}$:

$$x_k^{t+1} = \begin{cases} m_\kappa(l_i) - 1 & \text{если на шаге } t \text{ ядро } k \text{ выделено задаче } i, \\ \max(x_k^t - 1, 0) & \text{иначе.} \end{cases}$$

где $m_P(l) = l$, $m_E(l) = \lceil l\sigma \rceil$ — длина контракта в квантах именно на этом ядре.

[

Что означает « $l_i - 1$ », а не « l_i »? Первый квант контракта уже исполняется сейчас, на шаге t . К моменту $t + 1$ остаётся $l_i - 1$ квантов «аренды». Если $l_i = 1$ (односамплевая задача) — сразу $x_k^{t+1} = 0$, ядро свободно в следующем кванте.

[

Почему $\max(\cdot, 0)$? Отсчёт не уходит в минус. Когда контракт закончился ($x_k^t = 0$), ядро остаётся свободным и тик спустя ($x_k^{t+1} = 0$) — идемпотентно.

(2) Вектор бюджетов $\mathbf{b}^{t+1}$. Для каждой задачи i :

$$b_i^{t+1} = b_i^t - p_i^t + \Delta_i^t,$$

где p_i^t — VCG-платёж за квант (ноль, если задача не получила ядро), а $\Delta_i^t \geq 0$ — пополнение бюджета за время простоя (§I2 реализации: $\Delta_i^t = \Delta t_{\text{idle}} \cdot v_i / D_{\text{rep}}$).

[

Связь с §1.3] Условие $p_i \leq B_i$ — это инвариант, удерживаемый переходом: если p_i^t превысил бы остаток b_i^t , механизм в a^t не выделит задаче ядро (см. §8.3). То есть $b_i^{t+1} \geq 0$ всегда.

(3) Множество активных задач $\mathcal{N}^{t+1}$.

$$\mathcal{N}^{t+1} = (\mathcal{N}^t \setminus F^t) \cup N_{t+1},$$

где F^t — задачи, завершившиеся или отброшенные на шаге t (полностью отработали контракт, либо $b_i < 0$), а N_{t+1} — случайный i.i.d. набор новых задач.

(4) Внешние факторы ω^{t+1} эволюционируют по заданной цепи (температура, thermal throttling, память) — экзогенно к решению механизма.

Пример (один Р-ядерный квант). $K_P = 2$, $K_E = 0$, на шаге t :

$$\mathbf{x}^t = (0, 3), \quad \mathcal{N}^t = \{1, 2\}, \quad \theta_1 = (10, 2), \quad \theta_2 = (5, 4).$$

Ядро 1 свободно ($x_1^t = 0$), ядро 2 занято на 3 кванта. Механизм выбирает: выдать ядро 1 задаче 1 (у неё выше ϕ_P). Новых задач нет: $N_{t+1} = \emptyset$.

Тогда:

$$\mathbf{x}^{t+1} = (l_1 - 1, \max(3 - 1, 0)) = (1, 2), \quad \mathcal{N}^{t+1} = \{1, 2\}.$$

Задача 1 теперь «арендовала» ядро 1 ещё на 1 квант (итого её контракт $l_1 = 2$ квантов). Задача 2 продолжает досматривать свои 3 кванта на ядре 2.

Марковское свойство. s^{t+1} зависит только от s^t, a^t, N_{t+1} — не от истории. Это требование MDP: вся релевантная информация вшита в состояние. Именно поэтому $\mathbf{x}^t$ хранится явно (сколько осталось по контракту), а не выводится из прошлых решений.

[

Зачем внешние факторы ω] Они делают MDP реалистичным: термальный траслер меняет η_P, η_E , а значит и σ , а значит и ϕ_E . В коде это пока не используется (см. §I8 — σ берётся из статичного `max_car/min_car`), но в модели зафиксировано как `hook` для будущей динамики.

3.4 Вознаграждения (§2.4)

Задача-покупатель i :

$$r_i(s, a) = \begin{cases} v_i & \text{если } i \text{ получила ядро в первом кванте контракта,} \\ 0 & \text{иначе.} \end{cases}$$

Планировщик-продавец:

$$r_0(s, a) = \sum_{k \in \mathcal{P}} \mu_P \cdot \mathbb{I}[x_k \geq 1] + \sum_{k \in \mathcal{E}} \mu_E \cdot \mathbb{I}[x_k \geq 1], \quad \mu_P > \mu_E.$$

μ_k — базовая ставка вознаграждения за занятый квант.

[

Почему $\mu_P > \mu_E$] Р-ядро даёт больше «полезной работы» в тот же квант, значит платит больше seller-ренды. Это асимметрия, которая позже вшивается в $\phi_P \neq \phi_E$ через стоимости c_P, c_E .

4 Часть III. Контракты и ценообразование (s4, §3)

4.1 Простой контракт (§3.1)

$$z_i^t = (a_i, m_i, p_i, \kappa_i)$$

- $a_i \in [0, 1]$ — вероятность получить ядро;
- m_i — длина контракта: $m_i = l_i$ на P, $m_i = \lceil l_i \sigma \rceil$ на E;
- p_i — полный платёж;
- $\kappa_i \in \{P, E\}$ — тип ядра, выбранный механизмом.

«Простой» = раз выданный контракт гарантирован, без условий на будущее.

4.2 Стоимости ядер (§3.2)

$$c_P > c_E > 0, \quad \gamma = c_P/c_E > 1.$$

Полная стоимость контракта:

$$\text{cost}(i, P) = c_P \cdot l_i, \quad \text{cost}(i, E) = c_E \cdot \lceil l_i \cdot \sigma \rceil.$$

Какая дешевле — зависит от соотношения γ и σ :

$$\text{cost}(i, E) \leq \text{cost}(i, P) \iff \frac{\lceil l_i \sigma \rceil}{l_i} \leq \gamma.$$

[

σ против γ] σ — «во сколько раз медленнее Е», γ — «во сколько раз дороже Р за квант». Если $\sigma < \gamma$, Е-ядро дешевле даже с учётом замедления. Если $\sigma > \gamma$ — Р-ядро выгоднее. Именно поэтому роутинг не тривиален: нельзя просто «всё на Р».

4.3 Полезность задачи (§3.3)

$$u_i = \begin{cases} v_i - p_i & \text{при полном выполнении,} \\ -p_i & \text{иначе (прервана или не получила ядро),} \end{cases} \quad p_i \leq B_i.$$

5 Часть IV. Социальная функция стоимости (s4, §4)

5.1 Социальное благосостояние (§4.1)

Долгосрочное среднее:

$$W(\pi) = J(\pi; R) = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E} \left[\sum_{t=1}^T R(s^t, a^t) \right], \quad R = \sum_{j=0}^n r_j.$$

Это сумма всех вознаграждений (агентов и планировщика).

5.2 Социальная стоимость (§4.2)

Целевая функция оптимизации:

$$SC(\pi) = \underbrace{W(\pi)}_{\text{благосостояние}} - \underbrace{\lambda_F \cdot \Psi(\pi)}_{\text{штраф за неравенство}},$$

где

$$\Psi(\pi) = \text{Var}_i \left[\frac{u_i(\pi)}{v_i} \right].$$

[

Зачем нормировать на v_i] Без нормировки дорогая задача автоматически «стоит» больше и дисперсия завышается. Отношение u_i/v_i — доля от максимально возможной полезности; это честная метрика fairness.

[

Почему нет отдельного штрафа за энергию] Разница Е vs Р уже сидит в $c_P > c_E$: Е-ядра дешевле в квантовой стоимости. Добавлять ещё один энергетический член = двойной счёт.

6 Часть V. VCG-механизм (s4, §5)

6.1 Offline, полная информация (§5.1)

При известном MDP оптимум:

$$q^* = \arg \max_{q \in \Delta(P)} \langle q, R \rangle,$$

где q — осципация measure (доля времени в каждой паре «состояние–действие» под политикой).

Политика без задачи i :

$$q_{-i}^* = \arg \max_{q \in \Delta(P)} \langle q, R_{-i} \rangle, \quad R_{-i} = \sum_{j \neq i} r_j.$$

Платёж VCG:

$$p_i^*(s, a) = \underbrace{\langle q_{-i}^*, R_{-i} \rangle}_{\text{оптимум без } i} - \underbrace{R_{-i}(s, a)}_{\text{фактическая welfare без } i}.$$

[

Pivot-интуиция] Задача платит ровно экстерналию: «насколько хуже стало остальным из-за того, что я заняла ядро». Если моя задача никого не вытесняет, $p_i = 0$. Это ключ к incentive compatibility — врать про θ_i невыгодно, потому что платишь не свою ставку, а потерю соседей.

6.2 Адаптация к гетерогенным ядрам (§5.2)

Расширенная welfare через эффективную ценность:

$$W(s^t) = \max_{a^t} \left[\sum_{i \in \mathcal{N}^t} (a_{i,P}^t \phi_P(v_i, l_i) + a_{i,E}^t \phi_E(v_i, l_i)) + \delta \mathbb{E}[W(s^{t+1})] \right],$$

$$\boxed{\phi_P(v_i, l_i) = v_i - c_P \cdot l_i, \quad \phi_E(v_i, l_i) = v_i - c_E \cdot \lceil l_i \sigma \rceil.}$$

[

Смысл ϕ] ϕ_κ = «ценность минус стоимость занимаемого ресурса» = чистый вклад задачи в welfare на ядре типа κ . Механизм выбирает κ , максимизируя ϕ — не опрашивая задачу о её предпочтениях.

6.3 Виртуальные оценки (§5.3, Sano)

Для revenue-maximizing варианта:

$$\varphi(\theta_i) = v_i - \frac{1 - F(v_i | l_i)}{f(v_i | l_i)}$$

(Myerson virtual valuation). Оптимум:

$$R^*(s^t) = \max_{a^t} \left[\sum_i (a_{i,P}^t \varphi_P(\theta_i) + a_{i,E}^t \varphi_E(\theta_i)) + \delta \mathbb{E}[R^*(s^{t+1})] \right],$$

$$\varphi_P = \varphi - c_P l_i, \quad \varphi_E = \varphi - c_E \lceil l_i \sigma \rceil.$$

[

Условие регулярности] Monotone hazard rate: $\lambda_l(v) = f(v|l)/(1 - F(v|l))$ не убывает по v , не возрастает по l . Выполнено для большинства разумных распределений (экспоненциальное, лог-нормальное, Гамма с $k \geq 1$). Гарантирует монотонность оптимальной политики и, следовательно, её реализуемость как truthful механизма.

7 Часть VI. Протокол аукциона (s4, §6)

7.1 На каждом кванте (§6.1)

1. Планировщик наблюдает s^t .
2. Каждая задача $i \in \mathcal{N}^t$ подаёт ставку $b_i = (b_i^v, b_i^l)$ (объявленные v и l). Тип ядра не указывает — это решение аукциона.

3. Решается задача распределения:

$$a^t = \arg \max_a \sum_i [a_{i,P} \phi_P(b_i) + a_{i,E} \phi_E(b_i)] + \delta \mathbb{E}[W(s^{t+1})].$$

4. Считаются VCG-платежи p_i^t для победителей.

5. Обновление бюджетов $B_i \leftarrow B_i - p_i^t$.

6. Задачи с $B_i < 0$ отклоняются.

7.2 Платёж VCG (§6.2)

$$p_i^t = W_{-i}(s^t) - [W(s^t) - \phi_\kappa(\theta_i)] = W_{-i}(s^t) - W(s^t) + \phi_\kappa(\theta_i).$$

Частный случай (одно свободное ядро, по Sano):

$$p_i^t = \phi_\kappa(\hat{\theta}_j) + (\delta^{l_j} - \delta^{l_i}) \bar{W},$$

где j — задача на втором месте, $\bar{W}$ — ожидаемая будущая welfare.

[

Роль $\delta^{l_j} - \delta^{l_i}$] Этот член — «поправка на длину контракта»: победитель компенсирует разницу в том, насколько его контракт надолго блокирует ядро по сравнению с контрактом второго места. Длиннее контракт $\Rightarrow$ больше упущенная будущая welfare $\Rightarrow$ больше платит.

7.3 Дифференцированное ценообразование P vs E (§6.3)

$$p_P(l) = \frac{\delta - \delta^l}{1 - \delta^l} \bar{r}_P + \frac{1 - \delta}{1 - \delta^l} \cdot \frac{1}{\lambda(P^{**}(l))},$$

$$p_E(l) = \frac{\delta - \delta^{l'}}{1 - \delta^{l'}} \bar{r}_E + \frac{1 - \delta}{1 - \delta^{l'}} \cdot \frac{1}{\lambda(P^{**}(l'))}, \quad l' = \lceil l\sigma \rceil.$$

$\bar{r}_P > \bar{r}_E$ — средние выручки.

Свойство: $p_P(l) > p_E(l)$ за квант. При этом $P_E = p_E \cdot l' \leq P_P = p_P \cdot l$ — trade-off.

При независимости v_i и l_i (Sano, Prop.5): long-stay discount — $p_\kappa(l+1) < p_\kappa(l)$. Длинные задачи платят меньше за квант.

8 Часть VII. Онлайн-обучение (s4, §7)

Если распределения типов и переходы MDP неизвестны — учимся онлайн.

8.1 IHMDP-VCG (Leon & Etesami 2022)

Эпизоды $k = 1, 2, \dots$ растущей длины. В каждом эпизоде:

- Mixing phase, длина $d^{[k]} = \frac{\log k}{\alpha S}$ — без взимания платежей, цепь выходит на стационар.
- Stationary phase, длина $l^{[k]} = \frac{\max\{4, \sqrt{k}\} \log(|\mathcal{A}||\mathcal{S}|k/\zeta)}{\alpha \delta}$ — применяем $\pi^{[k]}$ и собираем платежи.

8.2 Обновление политики (§7.2)

Решаем $(n + 1)$ LP:

$$\hat{q}^{[k+1]} = \arg \max_{q \in \Delta_\delta(\hat{P}^{[k]})} \langle q, \hat{R}^{[k]} \rangle, \quad \hat{q}_{-i}^{[k+1]} = \arg \max_{q \in \Delta_\delta(\hat{P}^{[k]})} \langle q, \hat{R}_{-i}^{[k]} \rangle,$$

$$\hat{p}_i^{[k+1]}(s, a) = \langle \hat{q}_{-i}^{[k+1]}, \hat{R}_{-i}^{[k]} \rangle - \check{R}_{-i}^{[k]}(s, a).$$

$\hat{R}$ — UCB-оценка, $\check{R}$ — LCB. Это combines optimism (для allocation) с пессимизмом (для платежа) — уменьшает regret.

8.3 Гарантии (§7.3)

При $T \rightarrow \infty$ с вероятностью $\geq 1 - O(\zeta)$:

Метрика	Верх. граница regret
Welfare	$\tilde{O}(\varepsilon T n + T^{2/3} n)$
Seller	$\tilde{O}(\varepsilon T n^2 + T^{2/3} n^2)$
Bidder	$\tilde{O}(\varepsilon T n^2 + T^{2/3} n^2)$

Асимптотически: эффективность + truthfulness + IR одновременно.

9 Часть VIII. Свойства механизма (s4, §8)

9.1 Incentive compatibility (§8.1, Sano Thm 1)

Механизм стимульно-совместим тогда и только тогда, когда для каждой i :

1. $\alpha_i(v_i, l_i)$ не убывает по v_i при фикс. l_i .
2. $\int_0^{v_i} \alpha_i(\nu, l_i) d\nu$ не возрастает по l_i .
3. $\exists \underline{\Pi}_i$, независимое от l_i :

$$\Pi_i(v_i, l_i) = \underline{\Pi}_i + \int_0^{v_i} \alpha_i(\nu, l_i) d\nu.$$

Следствие: невыгодно завышать v_i , невыгодно занижать l_i .

9.2 IR (§8.2)

$$u_i(\pi^*, p^*) = W(\pi^*; R) - W(\pi_{-i}^*; R_{-i}) \geq 0.$$

Участие всегда выгодно.

9.3 Бюджетная допустимость (§8.3)

Дополнительное ограничение (отсутствует в классическом VCG):

$$p_i^*(s, a) \leq B_i^t \quad \forall t.$$

Если VCG-платёж превышает бюджет, задача не получает ядро в этом кванте:

$$\tilde{a}^t = \arg \max_a \sum_{i: p_i \leq B_i^t} [a_{i,P} \phi_P + a_{i,E} \phi_E] + \delta \mathbb{E}[W(s^{t+1})].$$

10 Часть IX. Пример: бинарные типы, одно свободное ядро (s4, §9)

Пусть $K_P = K_E = 1$ и свободно ядро типа κ .

Победитель:

$$i^* = \arg \max_{i \in \mathcal{N}^t, B_i^t > 0} \phi_\kappa(\theta_i) + \delta^{m_\kappa(l_i)} \bar{W}_\kappa,$$

где $m_P(l) = l$, $m_E(l) = \lceil l\sigma \rceil$.

Платёж:

$$p_{i^*} = \phi_\kappa^{-1} \left(\phi_\kappa(\theta_{j^*}) + (\delta^{m_\kappa(l_{j^*})} - \delta^{m_\kappa(l_{i^*})}) \bar{W}_\kappa \mid l_{i^*} \right),$$

j — второй по ранжированию.

Если свободны оба ядра:

$$\max_i \sum [a_{i,P} \phi_P + a_{i,E} \phi_E] + \delta \mathbb{E}[W(s^{t+1})]$$

при $a_{i,P} + a_{i,E} \leq 1$, $\sum a_{i,P} \leq 1$, $\sum a_{i,E} \leq 1$, $p_i \leq B_i^t$.

[

$\phi_P > \phi_E$ не всегда] При малом v_i и большом l_i : $c_E \cdot \lceil l_i \sigma \rceil < c_P \cdot l_i$ (если $\sigma < \gamma$) — Е-ядро может дать большую эффективную ценность. Вывод: самую ценную задачу на Р-ядро стоит пускать только когда выигрыш в скорости перевешивает разницу в стоимости.

10.1 Сводка обозначений

Символ	Значение
K_P, K_E	Число Р/Е ядер
$\mathcal{N}^t$	Активные задачи в момент t
$\theta_i = (v_i, l_i)$	Тип: ценность и длина
B_i	Бюджет задачи i
c_P, c_E	Стоимость кванта Р/Е, $c_P > c_E$
σ	Замедление Е-ядра, $\sigma = \eta_P / \eta_E > 1$
γ	Отношение стоимостей, $\gamma = c_P / c_E > 1$
ϕ_κ	Эффективная ценность на ядре κ
φ	Виртуальная оценка (Myerson)
p_i^*	VCG-платёж
$W(\pi)$	Социальное благосостояние
$SC(\pi)$	Социальная стоимость (с fairness-штрафом)
$q \in \Delta(P)$	Оссурансу measure
$m_\kappa(l)$	Длина контракта: $m_P = l$, $m_E = \lceil l\sigma \rceil$
$\delta \in (0, 1)$	Дисконт-фактор
$\rho_i = B_i / B_i^{\max}$	Бюджетное отношение (прокси «bursty vs CPU-bound»)
l_{ns}	EWMA длины активации (прокси l_i)

11 Часть X. Реализация: отличия от теории (VCG_s4_impl.md)

Абстрактная модель — канон. Ниже — что пришлось изменить в BPF-планировщике scx_auction и почему.

11.1 I1. Непрерывная ϕ (отказ от квантования)

Теоретическое ϕ считается в квантах:

$$\phi_P = v_i - c_P \lceil l_i/s \rceil, \quad \phi_E = v_i - c_E \lceil l_i \sigma/s \rceil.$$

Проблема: все задачи с $l_i < s$ дают $\lceil l_i/s \rceil = 1$ — короткие задачи неразличимы (потеря точности до $50\times$).

Реализация масштабирует v_i на s и использует нс:

$$\phi_P = v_i \cdot s - c_P \cdot l_{ns}, \quad \phi_E = v_i \cdot s - c_E \cdot l_{ns} \cdot \sigma,$$

l_{ns} — EWMA реально измеренного runtime (нс), $\sigma = \text{max_cap}/\text{min_cap}$.

[

Последствие: роутинг не зависит от l_{ns}] В линейной модели $\phi_P \geq \phi_E \iff c_E \sigma \geq c_P$. Это константа по задачам. Значит argmax по κ — чистое свойство платформы. Нужен другой сигнал роутинга — см. §I2.

11.2 I2. Роутинг по бюджетному отношению с гистерезисом

$$\rho_i = \frac{B_i}{B_i^{\max}} \in [0, 1].$$

Двусторонний гистерезис:

- $\rho_i \geq \rho_H$ (BURST_HIGH_PCT=70%) — bursty $\rightarrow$ AUCTION_DSQ_P.
- $\rho_i < \rho_L$ (BURST_LOW_PCT=30%) — CPU-bound $\rightarrow$ AUCTION_DSQ_E.
- $\rho_L \leq \rho_i < \rho_H$ — deadband, держим предыдущий выбор (tctx->on_p_type sticky).

Переоценка только на SCX_ENQ_WAKEUP. Preempt-переенкью держит прежний кластер.

[

Экономика ρ_i] Бюджет растёт пропорционально idle-времени ($\Delta t_{\text{idle}} \cdot v_i$). Значит ρ_i — отношение idle/active — прокси «насколько задаче срочно нужен быстрый ресурс». Большой ρ = интерактивная (пользователь ждёт). Маленький ρ = batch. Гистерезис = threshold policy with inaction region из одномерных контрактных аукционов: мелкие флуктуации не двигают задачу туда-обратно (anti-churn).

В презентации (21.04) показан упрощённый порог 50% без гистерезиса — это педагогическое упрощение. В коде настоящий 30/70. При вопросе из зала знай разницу.

11.3 I3. Пропорциональная задержка для «голодающих»

Задачи с $B_i < B_i^{\max}/\text{STARVE_FRAC}$ (порог 10) $\rightarrow$ в AUCTION_DSQ_STARVED. Виртуальный дедлайн сдвигается пропорционально дефициту ϕ :

$$v_d += \frac{\max(0, -\phi_\kappa)}{D_{\text{starve}}}, \quad D_{\text{starve}} = 1000.$$

Свойство: $\phi < 0$ (стоимость перевешивает ценность) $\rightarrow$ ждёт дольше. Глубже дефицит — дольше ждёт внутри starved. Это аппроксимация VCG-платежа: «дорогая» задача несёт большие последствия.

11.4 I4. Платёж VCG в форме Vickrey reserve

Теория (§6.2): $p_i^t = W_{-i}(s^t) - [W(s^t) - \phi_\kappa(\theta_i)]$. В одноядерном случае: $p_i \approx \phi_\kappa(\theta_j) + (\delta^{m_j} - \delta^{m_i})\bar{W}$. W_{-i} в BPF-хуке невыполнимо — нужен $O(1)$ приблизительно.

11.4.1 I4.1. Оценщик φ_{hi} на кластер

Глобальное поле на кластер κ . При каждом не-starved enqueue:

$$\varphi_{hi,\kappa} \leftarrow \begin{cases} \phi_\kappa(\theta_i) & \phi_\kappa > \varphi_{hi,\kappa}, \\ \frac{15 \varphi_{hi,\kappa} + \max(0, \phi_\kappa(\theta_i))}{16} & \text{иначе (EWMA, } \alpha = 1/16). \end{cases}$$

Climb-fast / decay-slow: новый максимум подхватывается мгновенно; без подтверждения плавно релаксирует к нулю (чтобы stale-всплеск не продолжал завышать платежи).

Почему это корректная аппроксимация] В стационарном режиме это сглаженная оценка максимума конкурирующих ϕ . По Myerson-regular-distribution для single-item — именно эта локальная статистика эквивалентна W_{-i} .

11.4.2 I4.2. Формула платежа

На auction_stopping:

$$p_i = \max\left(c_\kappa \cdot \frac{t_{\text{consumed}}}{s}, \quad \varphi_{hi,\kappa} \cdot \frac{t_{\text{consumed}}}{s^2}\right),$$

$s = \text{SCX_SLICE_DFL}$ (5 мс), $t_{\text{consumed}} = s - \text{p} \rightarrow \text{scx.slice}$.

Единицы совпадают] $\varphi_{hi,\kappa}$ имеет единицы $\text{weight} \cdot \text{ns}$ (после §I1). Делим на s^2 — получаем per-ns weight. Умножаем на t_{consumed} — weight-эквивалентный платёж за слайс. Тот же порядок, что и $c_\kappa \cdot t_{\text{consumed}}/s$.

Это Vickrey reserve form: c_κ играет роль Myerson reserve price (posted cost кластера), а pivot-терм поднимает платёж к экстерналии когда есть конкуренты.

11.4.3 I4.3. Свойства

- IR сохранено: пол $c_\kappa \cdot t/s$ гарантирует $p_i \geq \text{old posted-price}$. Не платит меньше, чем раньше.
- Cold-queue degeneracy: пустой кластер или только $\phi < 0 \rightarrow \varphi_{hi} \rightarrow 0 \rightarrow \text{возврат к posted-price}$. Старое поведение — zero-pivot limit.
- IC в пределе: при стационарности и monotone hazard rate φ_{hi} сходится к top-order статистике $\phi \rightarrow \text{механизм асимптотически truthful}$.
- Нет first-quantum penalty: оба терма $\propto t_{\text{consumed}}$.
- $O(1)$ в хуке: 1 load + 2 mul + 1 cmp. Нет скана очереди.

11.5 I5. Wake-latency boost (VCG бюджет $\leftrightarrow$ EEVDF дедлайн)

Стандартный EEVDF дедлайн:

$$v_d = v_e + \frac{q_{\text{max}} \cdot \text{SCALE}}{w_i}, \quad q_{\text{max}} = \text{max_cap} \cdot s / \text{CAPACITY_SCALE}.$$

w_i — вес (Linux nice).

На wake-up, если $\rho_i \geq \rho_H$ (высокий бюджет) — гар вдвое:

$$v_d = v_e + \frac{q_{\max} \cdot \text{SCALE}}{2 w_i}.$$

[

SC-интерпретация] Высокий $\rho_i = \varphi(\theta_i)$ выше недавних платежей $\Rightarrow u_i/v_i > 0$ и растёт. Пересмещаем задачу вперёд в dispatch-порядке — уменьшаем variance $\text{Var}_i[u_i/v_i]$, тем самым уменьшая fairness-штраф в $\text{SC}(\pi) = W - \lambda_F \Psi$. Это прямая реализация одного шага минимизации SC. Preempt-переенкью исключён (running-задача не «отстаёт»).

Свойства: boost ограничен (half-gap, не ниже v_e); budget-gated (CPU-bound не получают); revert при падении ρ . Не меняет ϕ , платёж, бюджет.

11.6 I6. Порядок dispatch и cross-cluster work-stealing

На каждом CPU при ops.dispatch():

local cluster DSQ $\succ$ cross-cluster DSQ $\succ$ AUCTION_DSQ_STARVED.

Cross-cluster work-stealing безусловен в обе стороны. Следует прямо из §1.1: idle = welfare loss. Р-ядро, жрущее Е-задачу, ускоряет её (усиливает realisation ϕ_E); Е-ядро с Р-задачей — subopt, но лучше чем пусто. Кластерные предпочтения держатся на этапе enqueue (§I2), не на dispatch.

11.7 I7. Work-conservation overrides

Три механизма, которые пробивают §I2 при наличии idle:

I7.1. Idle-wake local dispatch

ops.select_cpu() $\rightarrow$ scx_bpf_select_cpu_dfl() находит idle-CPU. Если кластер матчит preferred DSQ — задача идёт прямо в SCX_DSQ_LOCAL (минуя глобальные очереди). Иначе — один probe за idle-CPU нужного кластера; если нашёлся — swap; нет — запуск на «чужом» idle.

I7.2. P-cluster saturation spill

Если §I2 выбрал Р и

$$\text{scx_bpf_dsq_nr_queued}(\text{AUCTION_DSQ_P}) \geq K_P,$$

задача spill-ится в AUCTION_DSQ_E. Это enqueue-time enforcement $\sum_i a_{i,P} \leq K_P$. K_P — из /sys/devices/system/cpu/cpu*/cpu_capacity (90% threshold).

I7.3. Kick-on-enqueue

После scx_bpf_dsq_insert_vtime() — probe за idle-CPU и SCX_KICK_IDLE. Без этого deeply-idle CPU (особенно Е под Р-доминируемой нагрузкой) не запускают dispatch() до таймер-тика. Probe self-gating: нет idle $\rightarrow$ нет IPI.

11.8 I8. Параметры реализации

Константа	Значение	Смысл
C_P_DEF	512	c_P (override через -p)
C_E_DEF	256	c_E (override через -e)
BURST_HIGH_PCT	70	ρ_H — bursty $\rightarrow$ P
BURST_LOW_PCT	30	ρ_L — CPU-bound $\rightarrow$ E
BUDGET_MUL	2000	$B_i^{\max} = w_i \cdot \text{BUDGET_MUL}$
REPLENISH_DIV	$5 \cdot 10^6$	нс на единицу replenish
REPLENISH_IDLE_CAP	10^9	max idle interval (1 c)
STARVE_FRAC	10	$B_i < B_i^{\max}/10$
STARVE_PHI_DIV	1000	ϕ deficit $\rightarrow v_d$
P_CAP_PCT	90	P-ядро: $\geq 90\%$ max_cap
DISPATCH_BATCH_MAX	8	max tasks per dispatch
SCALE	100	vtime fixed-point
INV_SHIFT	20	inverse-weight shift

Runtime auction_ctx (из userspace): max_capacity, min_capacity, cost_p, cost_e, p_core_count, e_core_count, vtime_now, total_weight.

11.9 I9. Соответствие код $\rightarrow$ модель

Код	Символ / секция
p->scx.weight	v_i — §1.2
tctx->len_est_ns	l_{ns} — §I1
tctx->budget, budget_max	$B_i, B_i^{\max}$ — §1.3
gdata->cost_p/cost_e	c_P, c_E — §3.2
compute_phi()	ϕ_P, ϕ_E — §5.2, §I1
auction_enqueue() routing	a^t — §2.2, §I2, §I7
scx_bpf_dsq_insert_vtime(...vd)	EEVDF argmax $\approx$ §5.1
auction_stopping() budget update	p_i^t — §6.2, §I4
gdata->dsq_phi_hi_p/e	$\varphi_{hi,\kappa}$ — §I4.1
AUCTION_DSQ_STARVED	budget-feasibility — §8.3, §I3
running/stopping vtime	capacity-weighted vtime — §I1

12 Часть XI. Презентация 21.04 — по слайдам

Каждый слайд: что показано + что сказать (сжатая речь) + вероятные вопросы и ответы.

Слайд 1 — Титул

Показано: автор, тема, руководитель (Корхов В.В.), рецензент (Анохин Ю.В., НИЦ АЭРО-ДИСК).

Сказать: «Уважаемые члены комиссии, представляется бакалаврская работа на тему Математическое моделирование и реализация планировщика процессов для гетерогенных процессорных архитектур. Руководитель — к.ф.-м.н., доцент Корхов В.В.»

Слайд 2 — Постановка задачи

Показано: мотивация (гетерогенные CPU) + 2 цели (формализовать как VCG + реализовать через sched_ext).

Ключевая идея: ядро = perishable good, задача = agent with type $\theta_i = (v_i, l_i)$.

[

Q: почему EEVDF недостаточно для гетерогенных CPU? EEVDF — proportional-share: распределяет CPU-время по весам, но не знает про разницу стоимости/скорости между P и E. Выбор ядра в Linux делается sched_domains и EAS отдельно от основного алгоритма — они работают по эвристикам (utilization, capacity-aware load balancing), без единой оптимизационной модели. VCG даёт одну целевую функцию — социальное благосостояние, — и все решения (кто, куда, за сколько) выводятся из неё.

[

Q: почему именно Vickrey, не первая цена? Первая цена требует равновесной теории Байеса (знание распределений соперников) — невыполнимо для системы из тысяч процессов. VCG truthful в доминирующих стратегиях: задача оптимальна при любом поведении других.

Слайд 3 — Модель системы

Показано: $\mathcal{P}, \mathcal{E}$; дискретное время; $\sigma = \eta_P / \eta_E > 1$; общий ISA; тип $\theta_i = (v_i, l_i)$; полезность $u_i = v_i - p_i$ при full completion; таблица бюджетов по классам.

Подчеркнуть: ISA общий — любая задача исполнима на любом ядре, выбор — ход механизма.

[

Q: откуда приходит v_i для настоящего процесса Linux? В реализации $v_i \equiv p \rightarrow \text{scx.weight}$ — вес из nice-значения (nice -20 = 88761, nice 0 = 1024, nice 19 = 15). То есть v_i — administrator-supplied приоритет. Для будущих работ — учить v_i из ML-сигнала (interactivity score).

Слайд 4 — MDP модель

Показано: состояние $(\mathbf{x}, \mathbf{b}, \boldsymbol{\omega})$; действие $(a_{i,P}, a_{i,E})$ с ограничениями feasibility; вознаграждения $r_i = v_i \mathbb{1}[\text{finish}]$, r_0 (сервер); критерий $W(\pi)$.

Подчеркнуть: это average-cost MDP, не finite-horizon — поэтому у нас limit $1/T$.

[

Q: существование решения average-cost MDP?] Стандартная теория (Puterman, §8): достаточно weak accessibility — из любого состояния достижимо recurring state класса. В нашей модели: любая задача рано или поздно завершается или отбрасывается (budget exhaustion) $\Rightarrow$ цепь эргодична.

Слайд 5 — Механизм VCG

Показано:

$$\phi_P = v_i - c_P l_i, \quad \phi_E = v_i - c_E [l_i \sigma];$$

allocation $\operatorname{argmax} W$ с дисконтом δ ; pivot-платёж $p_i^t = W_{-i} - (W - \phi_\kappa)$; 3 свойства: IC, IR, budget.

Объяснить словами: «задача платит экстерналию» — 30 секунд максимум. Не уходить в измерение occupancy measure.

[

Q: как вы считаете W_{-i} в BPF?] Не считаю точно — это $O(|\mathcal{S}| \cdot |\mathcal{A}|)$, нереально в hot path. Вместо этого — Vickrey reserve form (§I4): бегущая оценка $\varphi_{hi,\kappa}$ на кластер с EWMA climb-fast / decay-slow. В пределе стационарной нагрузки она сходится к max-order statistic ϕ на кластере — той же локальной статистике, к которой W_{-i} сводится по теореме Myerson для single-item. Детали — слайд запасной / §I4 отчёта.

[

Q: что, если задача обанкротилась?] Секция §8.3: если $p_i^t > B_i^t$, задача не получает ядро в этот квант. В коде это AUCTION_DSQ_STARVED + пропорциональная задержка $v_d+ = \max(0, -\phi_\kappa)/D$. Не убиваем — демотивируем.

Слайд 6 — Маршрутизация

Показано: три DSQ — P, E, Starved. Порог $\rho^* = 50\%$ в презентации упрощён. Прирост бюджета за idle — $B_i+ = \Delta t_{\text{idle}} \cdot v_i / D_{\text{rep}}$.

[

Q: почему не один порог, а гистерезис в коде?] Чтобы избежать flapping между кластерами, когда ρ шумит около середины. В презентации для простоты показан один порог — в коде $\rho_H = 70\%$, $\rho_L = 30\%$ с deadband (§I2). Аналог — threshold policy with inaction region в 1D контрактных аукционах.

[

Q: связь с Myerson-порогом?] Прирост бюджета за idle = «виртуальное лобби» задачи: чем дольше ждала, тем больше competitive she becomes. Это дискретный аналог Myerson reserve: задача допускается к аукциону, как только её накопленный «виртуальный разряд» превосходит порог ρ^* .

Слайд 7 — sched_ext

Показано: eBPF как инфра ядра; DSQ (local/global/user); цикл select_cpu $\rightarrow$ enqueue $\rightarrow$ dispatch $\rightarrow$ running/stopping.

Подчеркнуть: верификатор eBPF гарантирует завершимость и memory-safety. Нет risk kernel-ranіs от багов.

[

Q: какой оверхед sched_ext vs нативный?] По LWN/2408.01997: порядка единиц процентов на micro-benchmarks, часто compensated за счёт лучших решений (как у нас на hackbench). В нашей таблице (slide 11) scx_EEVDF (просто перенос в BPF) на sysbench быстрее нативного EEVDF — разница в пределах шума.

Слайд 8 — Среда тестирования

Показано: Intel Core Ultra 7 265K (8P + 12E), Linux 7.0; 4 сравниваемых планировщика (Linux EEVDF, scx_EEVDF, scx_LAVD, scx_auction); 3 бенчмарка (hackbench, sysbench, sched_latency); метрики (время, events/s, p99, RAPL).

[

Q: почему именно эти 4 планировщика?] Linux EEVDF — baseline (mainline). scx_EEVDF — контрольный перенос (отделяет оверхед BPF от разницы алгоритмов). scx_LAVD — state-of-the-art на латентность, специализирован под interactive. scx_auction — наш. Четыре точки образуют осмысленное сравнение по оси «простота ↔ специализация».

Слайд 9 — Hackbench

Показано: scx_auction — лучшее время (8.43s vs 9.27s у native EEVDF) — примерно 9% выигрыша.

Объяснение: интерактивные задачи с высоким ρ сразу идут в DSQ_P (минуя глобальную очередь + wake-boost из §15). Hackbench — чистый socket-ping-pong, доминирует wake-up latency.

Слайд 10 — Sysbench

Показано: scx_auction отстаёт от лидера (scx_EEVDF) на $\approx 4.3\%$.

Объяснение: CPU-bound нагрузка $\rightarrow$ бюджеты быстро тают $\rightarrow \rho < \rho_L \rightarrow$ задачи на E-ядра. Это сознательный trade-off (sacrifice peak throughput for energy).

[

Q: можно ли выключить миграцию в sysbench?] Да — CLI-параметр -e c_E с большим значением делает E-ядра дорогими, и роутинг начинает предпочитать P. Но тогда теряется смысл гетерогенного плана. Лучше — tune c_P/c_E под workload profile.

Слайд 11 — Итоговая таблица

Показано: 8 метрик $\times$ 4 планировщика. Победы scx_auction: hackbench, sleep duration. Проигрывши: sched delay p99 (LAVD лучше), sysbench (scx_EEVDF чуть лучше).

Ключевой вывод: VCG-подход конкурентоспособен на throughput + адекватен по энергии, но требует доработки tail-latency.

[

Q: почему p99 plot хуже 38ms?] Две причины: (1) STARVED-очередь по построению демонстрирует задачу ценой задержки — это by design, fairness-штраф; (2) wake-boost половинит гар только для bursty — CPU-bound задачи иногда ждут. LAVD оптимизирован именно

под p99 и использует другие эвристики (vdeadline scaling, lock holder boost).

[

Q: context switch K/s 62 vs 187 — почему так мало?] Аукцион группирует исполнение в длинные slice-ы: раз задача выиграла, она докручивает контракт (§3.1 «simple contracts» — full length guaranteed). Это снижает миграции и preempt-переключения.

Слайд 12 — Литература

Ключевые ссылки:

- Leon, Etesami 2022 (NeurIPS) — IHMDP-VCG и онлайн-обучение.
- Sano 2023 — dynamic mechanism design, многопериодные контракты, Thm 1 (IC).
- Myerson 1981 — optimal auction design, virtual valuation.
- Stoica–Abdel-Wahab 1996 — EEVDF.
- Agrawal et al. 2024 (arXiv:2408.01997) — sched_ext.
- ghOSt 2021 (SOSP) — user-space scheduling, идейный аналог.
- PIE 2012 (ISCA) — performance impact estimation для hetero-scheduling.

Extra-1 — IC теорема (Sano Thm 1)

Три условия (см. §VIII.1). Практическое следствие: «врать про v_i или l_i невыгодно».

Extra-2 — IHMDP-VCG онлайн

Эпизоды: mixing + stationary. Regret $\tilde{O}(\varepsilon T n + T^{2/3} n)$ для welfare; квадратично по n для seller/bidder. См. §VII.

Extra-3 — Пример $K_P = K_E = 1$

Победитель по $\phi_\kappa + \delta^{m_\kappa(l_i)} \bar{W}$; платёж через ϕ_κ^{-1} от второго места. Подчёркивает нетривиальность P-vs-E выбора.

13 Часть XII. Частые вопросы комиссии — шпаргалка

Q: “Это же просто EEVDF с новыми весами?”] Нет. EEVDF = proportional-share: w_i фиксирован и задача получает $\sum w_j$ -долю CPU. У нас v_i — это оценка, а p_i^t — платёж, и бюджет B_i истощается. Если задача спамит CPU — она платит и теряет приоритет. EEVDF такого механизма не имеет. Плюс — у EEVDF нет модели P/E, у нас $\phi_P \neq \phi_E$ и явный выбор.

Q: “Откуда известна истинная v_i ? Процесс ведь не ставит ставку.”] В текущей реализации $v_i = w_i$ (weight из nice). Это администратор-supplied значение — кому доверяют столько, тот и имеет такой вес. Механизм truthful в том смысле, что если nice отражает настоящий приоритет, аукционное распределение будет социально-оптимальным. Будущая работа — автоматическая оценка v_i из сигналов (interactivity, latency sensitivity).

Q: “Complexity per-tick?”] В hot-path (enqueue, stopping): $O(1)$. Нет скана очереди, нет решения LP. Все решения — локальные: ϕ -comparison, ρ -test, EWMA update. Дорогие операции (периодический replenish, metrics) — throttled to per-task periods ~ 1 ms.

Q: “А как же starvation freedom?”] Гарантии starvation freedom через: (1) bounded waiting через STARVE_FRAC и пропорциональную задержку; (2) replenish бюджета за idle; (3) work-conservation (§I6, §I7) — idle ядро не стоит пустым.

Q: “Почему social cost $W - \lambda_F \Psi$, а не multi-objective?”] Scalarization проще (одна скалярная MDP), и λ_F — явный tunable. Pareto-frontier исследование — future work.

Q: “Как выбираются c_P, c_E ?”] Defaults 512 / 256 откалиброваны на Intel Core Ultra 7 265K: $\max_capacity / \min_capacity \approx 2$, и $\gamma = c_P / c_E = 2$ даёт сбалансированный роутинг. CLI -p/-e override. Для другой платформы — tune.

Q: “Что, если workload меняется?”] Адаптация через (а) EWMA l_{ns} — длина обновляется; (б) bundle-decay φ_{hi} — старые pivot-оценки растворяются; (в) ρ -гистерезис — роутинг переоценивается на каждом wake. Формальный анализ — IHMDP-VCG regret bounds (§VII.3).

Q: “Кернел нестабилен с BPF-планировщиком?”] sched_ext включает scx_ops_error_fallback: при сбое BPF-программы ядро возвращается к CFS автоматически. Плюс eBPF verifier отклоняет программу, которая может зациклиться или выйти за память. В experimentation ни одного kernel panic за > 100 h нагрузки.

Приложение А. Чек-лист перед защитой

- ☐ Знать формулу $\phi_P = v - c_P l$, $\phi_E = v - c_E[l\sigma]$.
- ☐ Знать формулу pivot $p_i = W_{-i} - (W - \phi_\kappa)$.
- ☐ Помнить, что в коде два порога (30/70), в слайдах один (50).
- ☐ Объяснить VCG на пальцах за 30 сек («экстерналия»).
- ☐ Назвать три свойства: IC, IR, budget feasibility.
- ☐ Знать, что $v_i \equiv p \rightarrow \text{scx.weight}$ (nice).
- ☐ Связь wake-boost $\leftrightarrow$ SC fairness-term.
- ☐ Почему scx_auction хуже на sysbench — sacrifice for energy.
- ☐ Почему лучше на hackbench — DSQ_P bypass + wake-boost.
- ☐ LAVD лучше по p99 — оптимизирован именно под это.
- ☐ На вопрос «как считаете W_{-i} » — ответ «Vickrey reserve form, EWMA».